Aula 24
Grafos

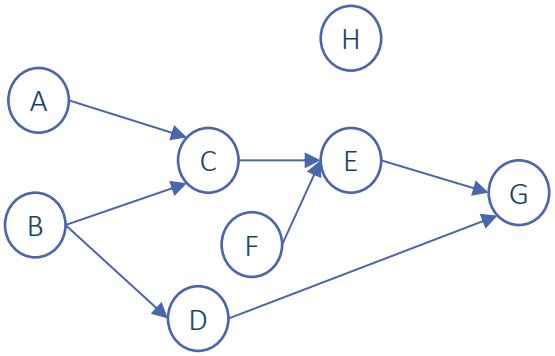
Algoritmos sobre Grafos (Parte II)
Algoritmos e Estruturas de Dados

Ordenação

Topológica

Problemas de calendarização

- Problemas de calendarização podem ser representados através de Grafos
  - Vértices representam tarefas
  - Arcos representam restrições de dependências



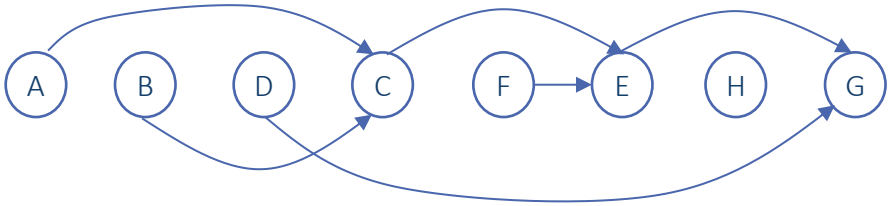
  - Problema de calendarização pode ser resolvido através de uma ordenação topológica

Ordenação Topológica

- Ordenação Topológica
  - Dado um Grafo Dirigido G, encontrar uma ordenação dos vértices tal que:

    para todos os arcos dirigidas de G: $v \rightarrow w$

    $v \prec w$ *de acordo com a ordenação*



Ordenação topológica para o Grafo do slide anterior.
Todos os arcos vão da esquerda para a direita.

Observação:
Um Grafo dirigido tem uma ordenação topológica sse for um DAG
(isto é, não tem ciclos)

Ordenação Topológica

- Uma ordenação topológica pode ser obtida a partir de uma procura em profundidade primeiro
  - Usando a pós-ordem invertida da procura.

  - Numa procura em profundidade primeiro:

    A pré-ordem dá-nos a ordem pela qual os nós são visitados

    A pós-ordem dá-nos a ordem pela qual terminam de ser visitados

Implementação Ordenação Topológica

```
public class RecursiveDFS {
      private boolean[] visited;
      private int count;
      private DiGraph graph;
      private Queue<Integer> preOrder;
      private Queue<Integer> postOrder;
      private Stack<Integer> reversePostOrder;

      public RecursiveDFS(DiGraph g)
      {
            this.graph = g;
            this.visited = new boolean[g.vCount()];
      }

      public void search()
      {
            int vertices = this.graph.vCount();
            for(int i = 0; i<vertices;i++){this.visited[i] = false;}

            this.preOrder = new ArrayDeque<Integer>();
            this.postOrder = new ArrayDeque<Integer>();
            this.reversePostOrder = new Stack<Integer>();

            this.count = 0;

            for(int i = 0; i < vertices; i++)
            {
                  if(!this.visited[i]) visit(i);
            }
      }
```

1.º Passo
Rever a implementação da procura em profundidade primeiro para passar a calcular:
Pré-ordem
Pós-ordem
Pós-ordem invertida

Observação:
Vamos iniciar uma procura para cada vértice não visitado, para o caso do Grafo ter múltiplos componentes.

Caso só haja um componente, apenas vai ser feita a 1.ª procura.

Implementação Ordenação Topológica

```
private void visit(int v)
{
      this.count++;
      this.visited[v] = true;
      this.preOrder.add(v);

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }

      this.postOrder.add(v);
      this.reversePostOrder.add(v);
}

public Iterable<Integer> preOrder(){return this.preOrder;}

public Iterable<Integer> postOrder(){return this.postOrder;}

public Iterable<Integer> reversePostOrder(){return this.reversePostOrder;}
```

Observação:

- A pré-ordem dá-nos a ordem pela qual os vértices são visitados
- A pós-ordem dá-nos a ordem pela qual terminam de ser visitados

Lembrete:

Quando queremos inverter a ordem basta usar uma Pilha, pois na pilha o último a entrar é o primeiro a sair

Implementação Ordenação Topológica

```
public class TopologicalOrder {
	private boolean hasTopologicalOrder;
	private DiGraph graph;
	Iterable<Integer> topologicalOrder;

	public TopologicalOrder(DiGraph g)
	{
		this.graph = g;
		this.hasTopologicalOrder = false;
	}

	public void determineTopologicalOrder()
	{
		CycleDetector detector = new CycleDetector(this.graph);
		detector.search();
		if(!detector.hasCycle())
		{
			RecursiveDFS dfs = new RecursiveDFS(this.graph);
			dfs.search();

			this.hasTopologicalOrder = true;
			this.topologicalOrder = dfs.reversePostOrder();
		}
		else this.hasTopologicalOrder = false;
	}
	public boolean hasTopologicalOrder() {return this.hasTopologicalOrder;}

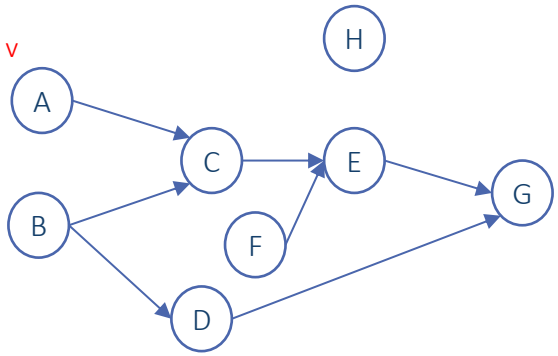
	public Iterable<Integer> order() {return this.topologicalOrder;}
```

2.º Passo – Algoritmo Topological Order

Começamos por detetar se existem ciclos

Caso não existam usamos uma procura em profundidade primeiro, e extraímos a pós-ordem invertida

Exemplo Ordenação Topológica



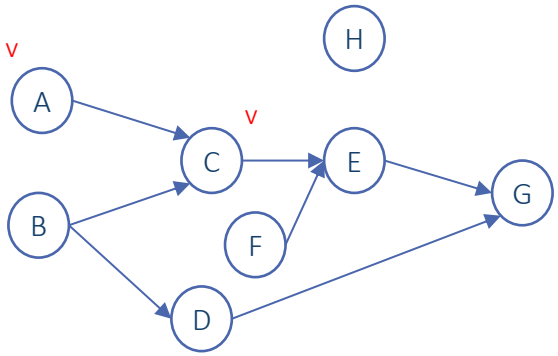
Call stack

search()
visit(A)
visit(C)

for([C])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A		

Exemplo Ordenação Topológica

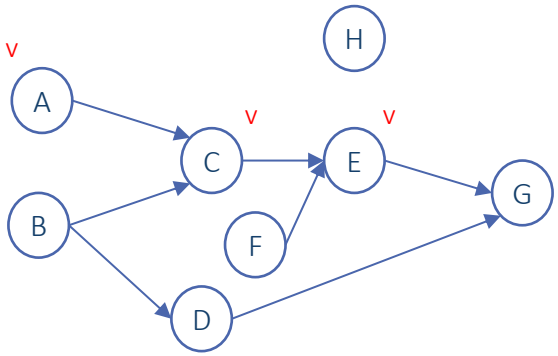


Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C		

Exemplo Ordenação Topológica

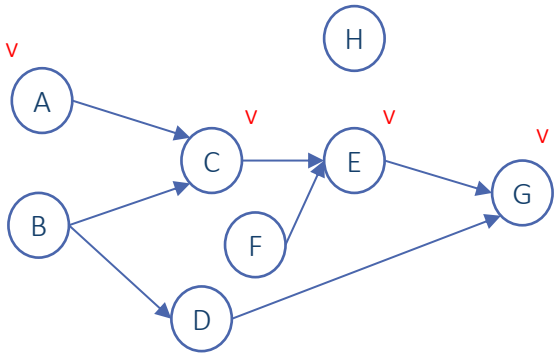


Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E		

Exemplo Ordenação Topológica

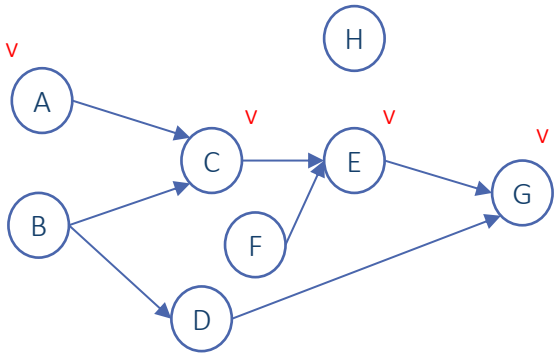


Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E **G**		

Exemplo Ordenação Topológica

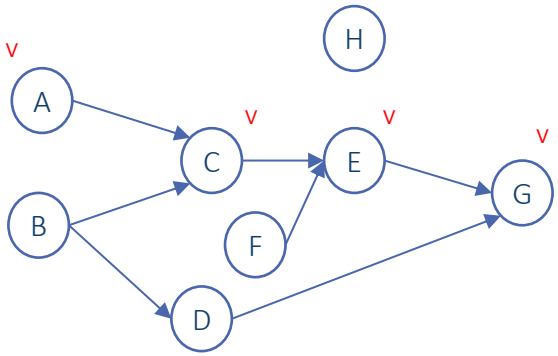


Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G	G	G

Exemplo Ordenação Topológica

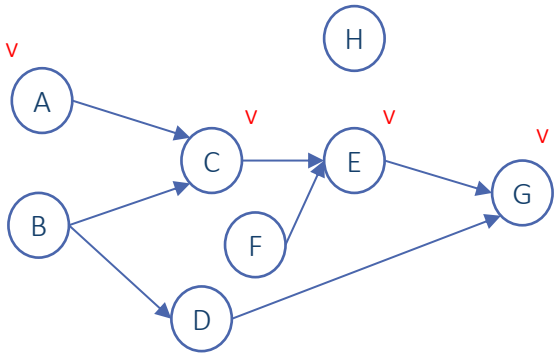


Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G	G **E**	**E** G

Exemplo Ordenação Topológica

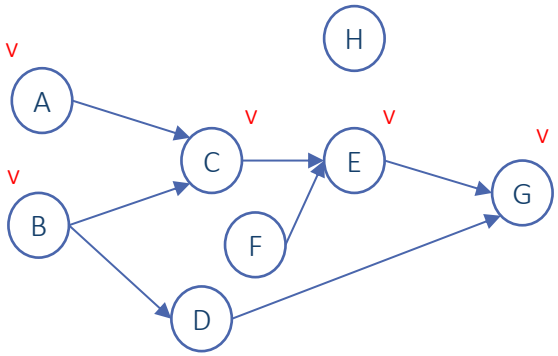


Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G	G E C A	A C E G

Exemplo Ordenação Topológica



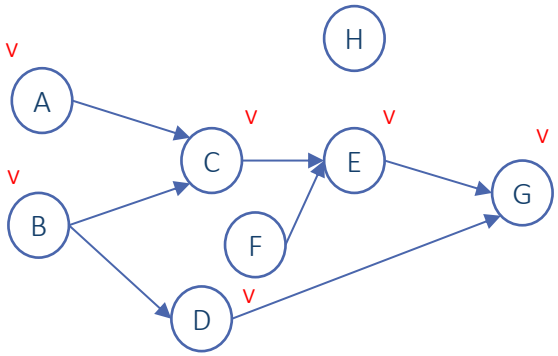
Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

visit(B) for([C,D])
visit(D)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G **B**	G E C A	A C E G

Exemplo Ordenação Topológica



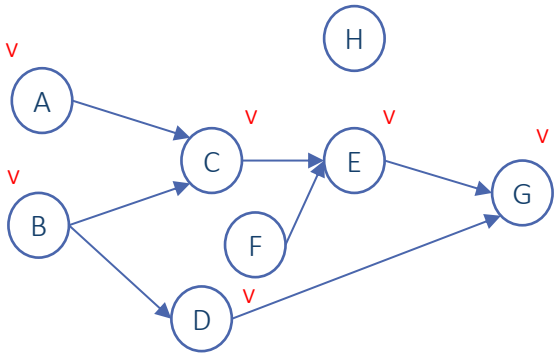
Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

visit(B) for([C,D])
visit(D) for([G])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B **D**	G E C A	A C E G

Exemplo Ordenação Topológica



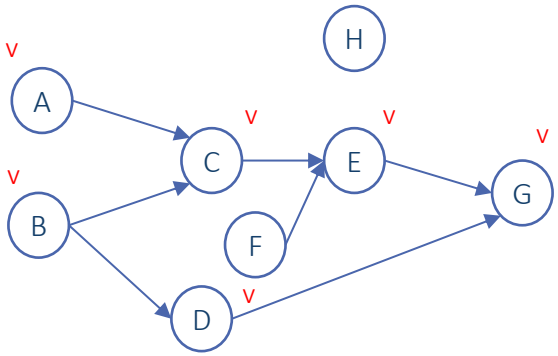
Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

visit(B) for([C,D])
visit(D) for([G])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B D	G E C A D	D A C E G

Exemplo Ordenação Topológica



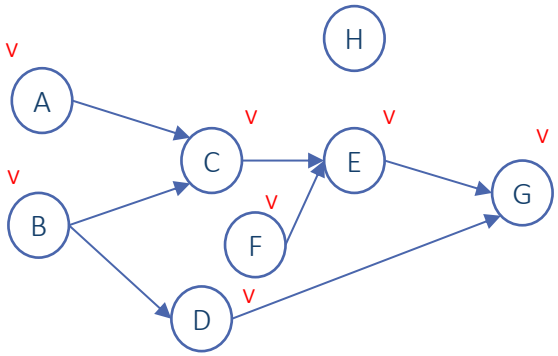
Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

visit(B) for([C,D])
visit(D) for([G])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B D	G E C A D **B**	**B** D A C E G

Exemplo Ordenação Topológica



Call stack

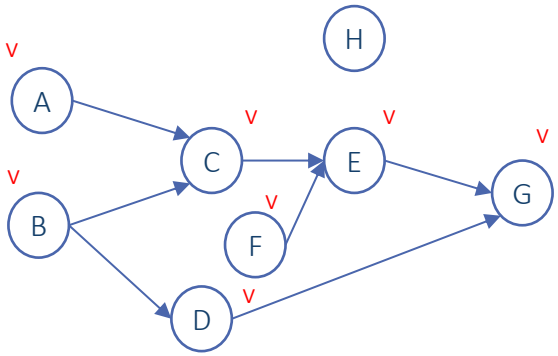
search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

visit(B) for([C,D])
visit(D) for([G])

visit(F) for([E])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B D F	G E C A D B	B D A C E G

Exemplo Ordenação Topológica



Call stack

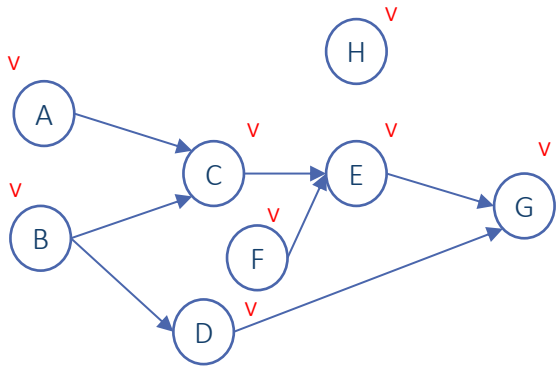
search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

visit(B) for([C,D])
visit(D) for([G])

visit(F) for([E])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B D F	G E C A D B F	F B D A C E G

Exemplo Ordenação Topológica



Call stack

search()
visit(A) for([C])
visit(C) for([E])
visit(E) for([G])
visit(G) for()

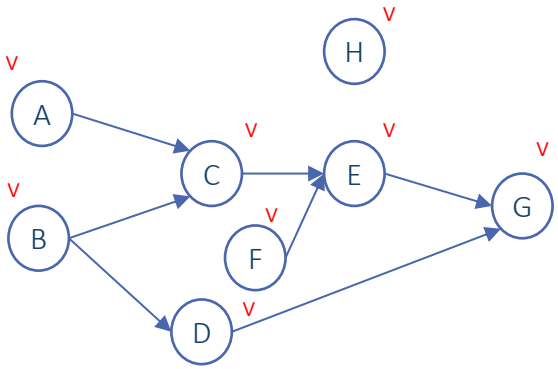
visit(B) for([C,D])
visit(D) for([G])

visit(F) for([E])

visit(H) for([])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B D F H	G E C A D B F	F B D A C E G

Exemplo Ordenação Topológica



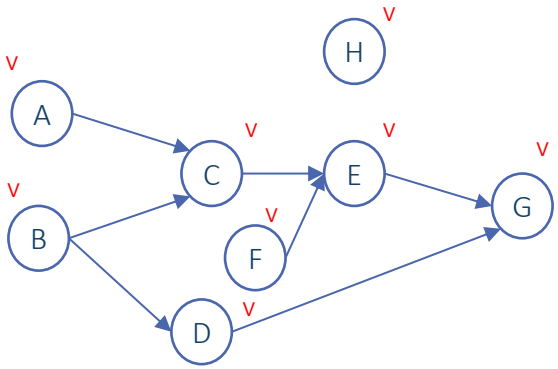
Call stack

search()

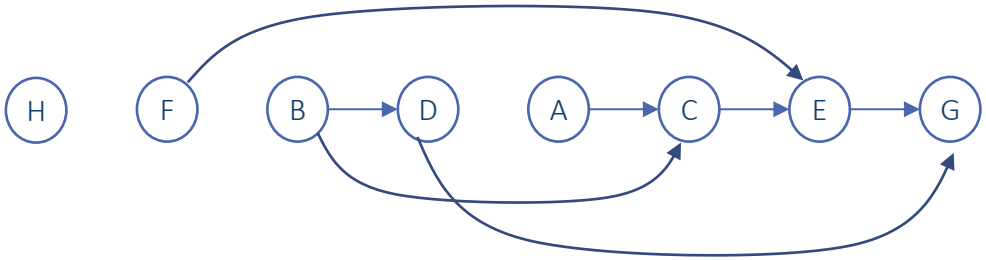
visit(A)	for([C])
visit(C)	for([E])
visit(E)	for([G])
visit(G)	for()
visit(B)	for([C,D])
visit(D)	for([G])
visit(F)	for([E])
visit(H)	for([])

Pré-ordem	Pós-ordem	Pós-ordem invertida
A C E G B D F H	G E C A D B F H	H F B D A C E G

Exemplo Ordenação Topológica



Pós-ordem invertida
H F B D A C E G



A pós-ordem invertida corresponde efetivamente a uma ordenação topológica válida.
Todos os arcos vão da esquerda para a direita.

Complexidade Temporal

- Algoritmo de Ordenação Topológica corresponde a
  - Procura em profundidade primeiro – deteção de ciclos
  - Procura em profundidade primeiro – determinar a ordenação

  - $T_{topológica} = 2(E+V)$

  - Na realidade, é possível fazer a deteção de ciclo na mesma ppp que determina a ordenação

  - $T_{topológica} = (E+V)$

Identificar

Componentes fortemente ligados

Componentes fortemente ligados

- Problema:
- Dado um grafo dirigido G, como determinar o número de componentes fortemente ligados, e quais os vértices que correspondem a cada componente?
- Algoritmo de Kosaraju-Sharir

Relembrar:

Vértices fortemente ligados: dois vértices v e w dizem-se fortemente ligados sse forem mutuamente alcançáveis. Ou seja, existe um caminho dirigido de v para w, e um caminho dirigido de w para v.

Grafo fortemente ligado: um grafo dirigido diz-se fortemente ligado se todos os seus vértices são fortemente ligados entre si.

Um grafo dirigido que não seja fortemente ligado é composto por um conjunto de componentes fortemente ligados, que por sua vez são subgrafos dirigidos fortemente ligados.

Algoritmo de Kosaraju-Sharir

- Determina o número de componentes fortemente ligados e os vértices correspondentes a cada um
  - Dado um Grafo dirigido G:

    1) calcular o Grafo invertido G^R

    2) usar uma ppp para determinar a pós-ordem invertida de G^R :$po^R(G^R)$

    3) visitar todos os vértices de G com uma ppp, mas usando a ordem $po^R(G^R)$

    a) todos os vértices visitados em cada uma das procuras de 3) correspondem a um componente fortemente ligado

Algoritmo de Kosaraju-Sharir

```
public class KosarajuSharir {
      private boolean[] visited;
      private int[] componentOfVertex;
      private int componentCounter;
      private DiGraph graph;

      public KosarajuSharir(DiGraph g) {
            this.graph = g;
            this.visited = new boolean[g.vCount()];
            this.componentOfVertex = new int[g.vCount()];
      }

      public void determineSCC() {
            int vertices = this.graph.vCount();
            this.componentCounter = 0;
            for(int i = 0; i<vertices;i++)
            {
                  this.visited[i] = false;
                  this.componentOfVertex[i] = 0;
            }

            DiGraph GR = this.graph.reverse();
            RecursiveDFS dfs = new RecursiveDFS(GR);
            dfs.search();
            for(int v : dfs.reversePostOrder())
            {
                  if(!this.visited[v]) {visit(v); this.componentCounter++;}
            }
      }
```

1 – calcular o grafo com ordem inversa G^R

2 – determinar pós-ordem inversa de G^R

3 – Usar a ordem para visitar todos os vértices de G

Cada vez que saímos de uma procura e não visitamos todos os nós, sabemos que temos pelo menos mais um componente

Algoritmo de Kosaraju-Sharir

```
private void visit(int v)
{
      this.visited[v] = true;
      this.componentOfVertex[v] = this.componentCounter;

      for(int adj : graph.adj(v))
      {
            if(!visited[adj]) visit(adj);
      }
}

public boolean isStronglyConnected(int v, int w)
{
      //two vertices are strongly connected if they belong to the same component
      return this.componentOfVertex[v] == this.componentOfVertex[w];
}

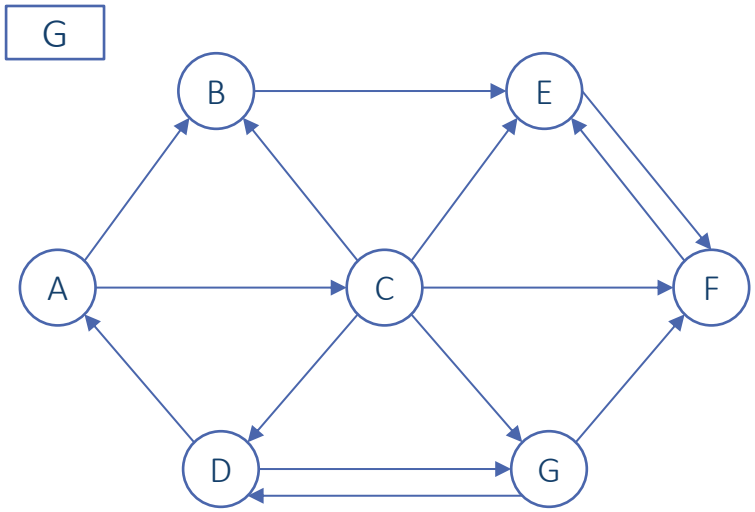
public int numberOfStrongComponents()
{
      return this.componentCounter;
}

public int getComponentOfVertex(int v)
{
      return this.componentOfVertex[v];
}
```

Quando um vértice é visitado pela 1.ª vez, guardamos qual o componente ao qual ele pertence.

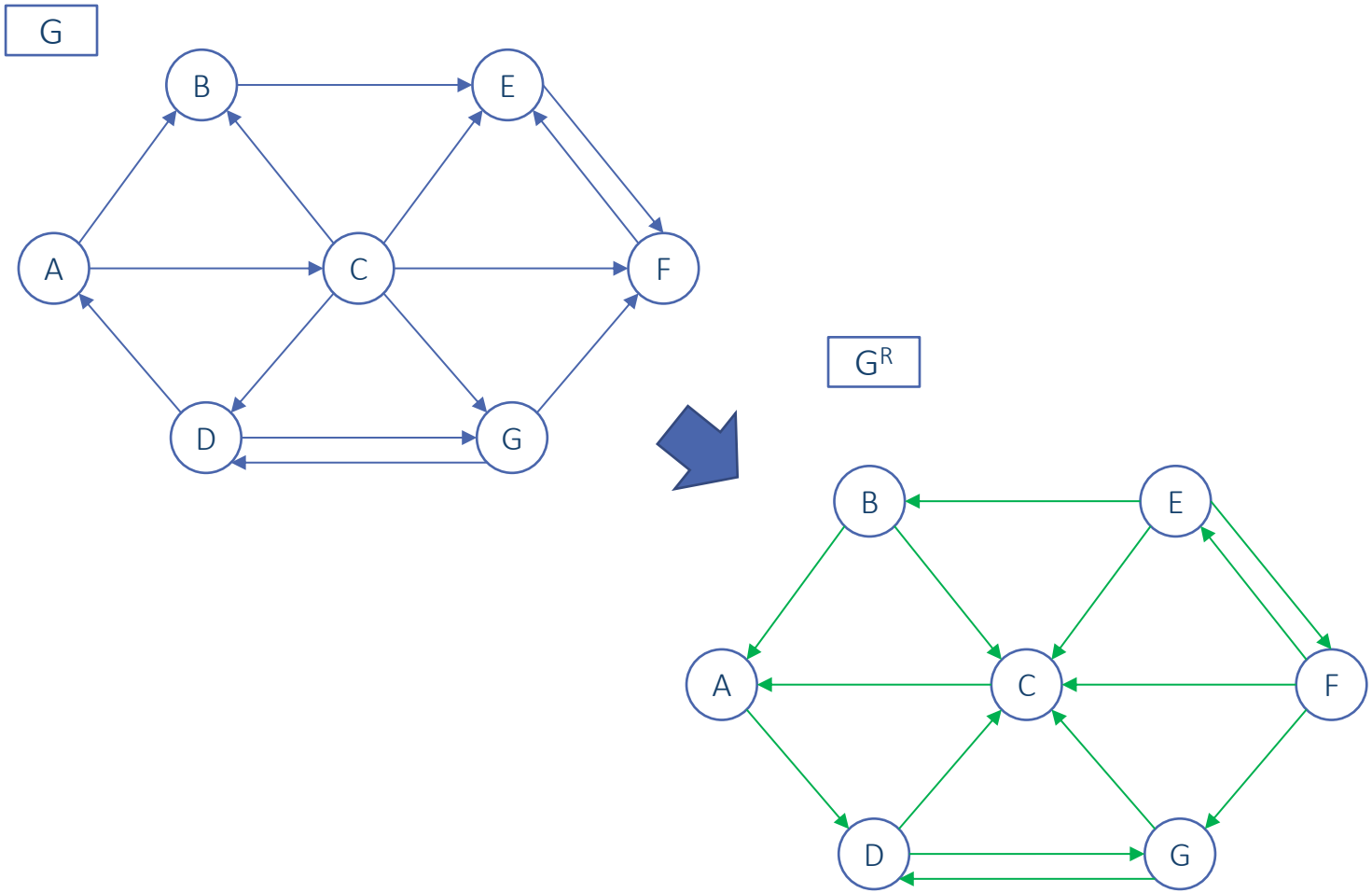
Um vértice visitado na 1.ª procura pertence ao componente 0.
Um vértice visitado na 2.ª procura pertence ao componente 1…

Exemplo



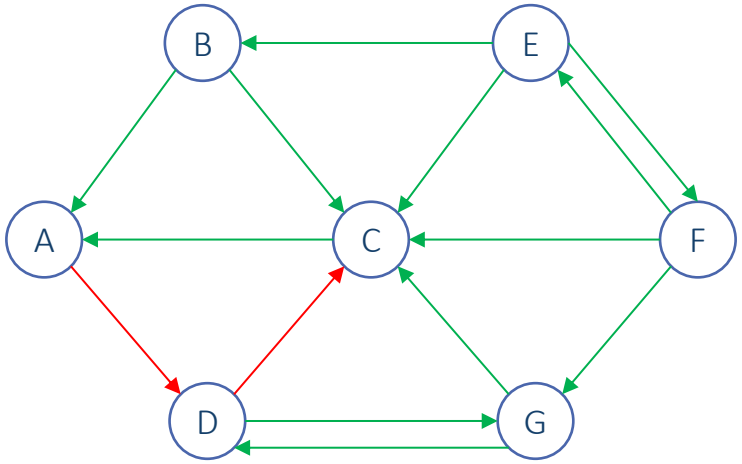
Indique o número de componentes fortemente ligados do grafo

Exemplo



Exemplo

G^R



Fazer uma ppp, usando a ordem alfabética dos nós

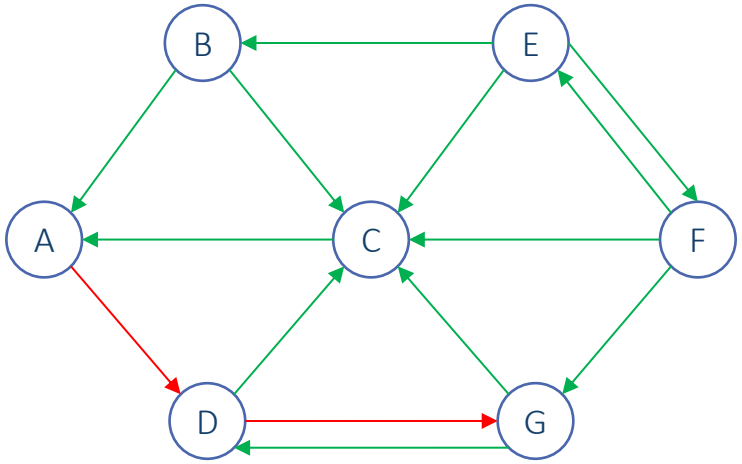
Call stack

search()
  visit(A)
    visit(D)
      visit(C)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A D C		

Exemplo

G^R



Fazer uma ppp, usando a ordem alfabética dos nós

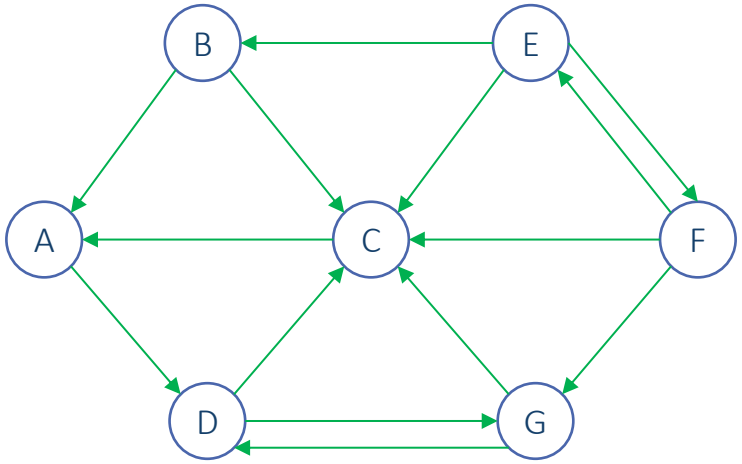
Call stack

search()
visit(A)
visit(D)
visit(C)
visit(C)
visit(G)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A D C G	C	C

Exemplo

G^R



Fazer uma ppp, usando a ordem alfabética dos nós

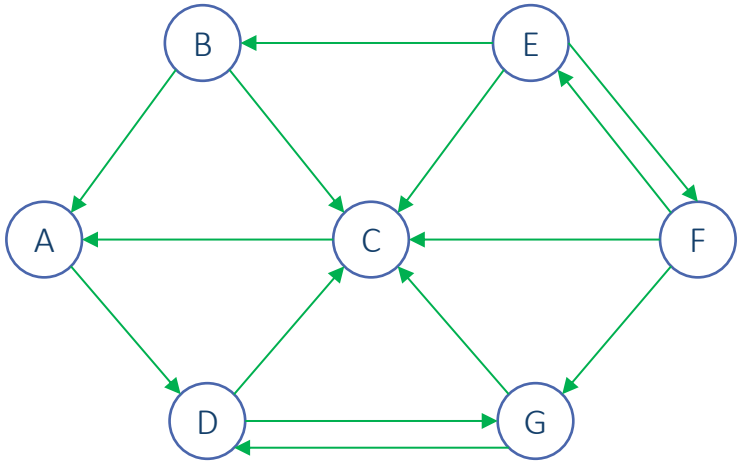
Call stack

search()
visit(A)
visit(D)
visit(C)
visit(C)
visit(G)
visit(G)
visit(D)
visit(A)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A D C G	C G D A	A D G C

Exemplo

G^R



Fazer uma ppp, usando a ordem alfabética dos nós

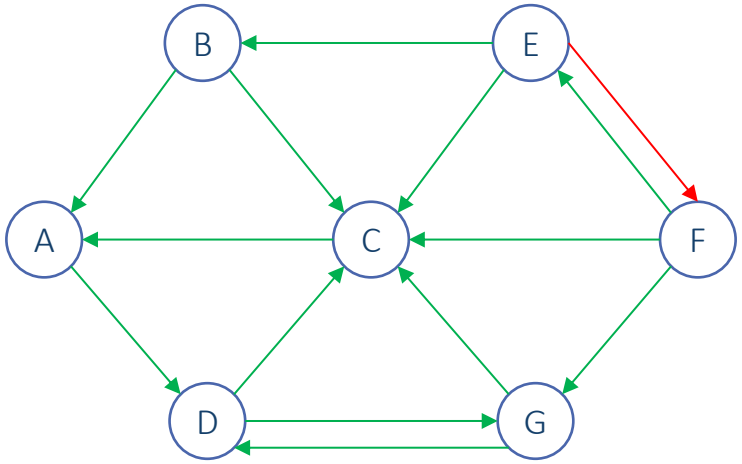
Call stack

search()
visit(A)
....
visit(A)
visit(B)
visit(B)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A D C G B	C G D A B	B A D G C

Exemplo

G^R



Fazer uma ppp, usando a ordem alfabética dos nós

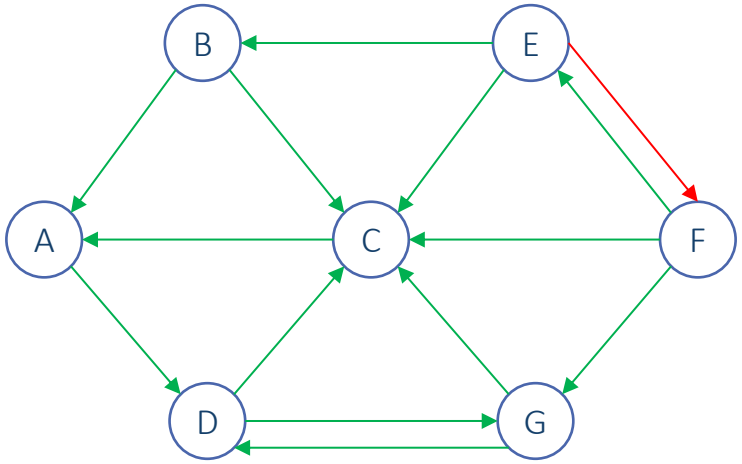
Call stack

search()
visit(A)
....
visit(A)
visit(B)
visit(B)
visit(E)
visit(F)

Pré-ordem	Pós-ordem	Pós-ordem invertida
A D C G B E F	C G D A B	B A D G C

Exemplo

G^R



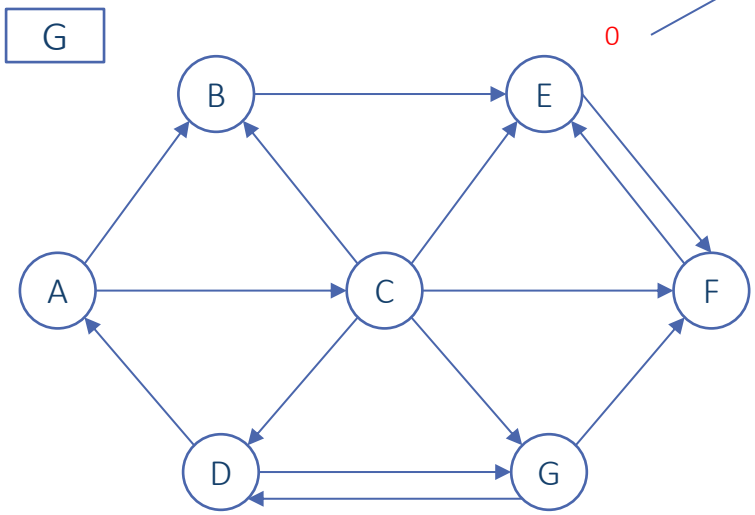
Fazer uma ppp, usando a ordem alfabética dos nós

Call stack

search()
visit(A)
....
visit(A)
visit(B)
visit(B)
visit(E)
visit(F)
visit(F)
visit(E)
search()

Pré-ordem	Pós-ordem	Pós-ordem invertida
A D C G B E F	C G D A B F E	E F B A D G C

Exemplo

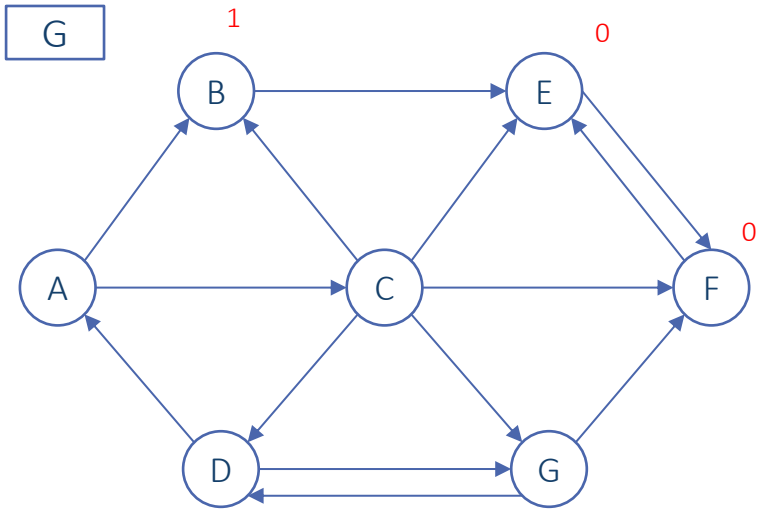


Pós-ordem invertida G^R
E F B A D G C

Call stack

search()
visit(E) 0

Exemplo

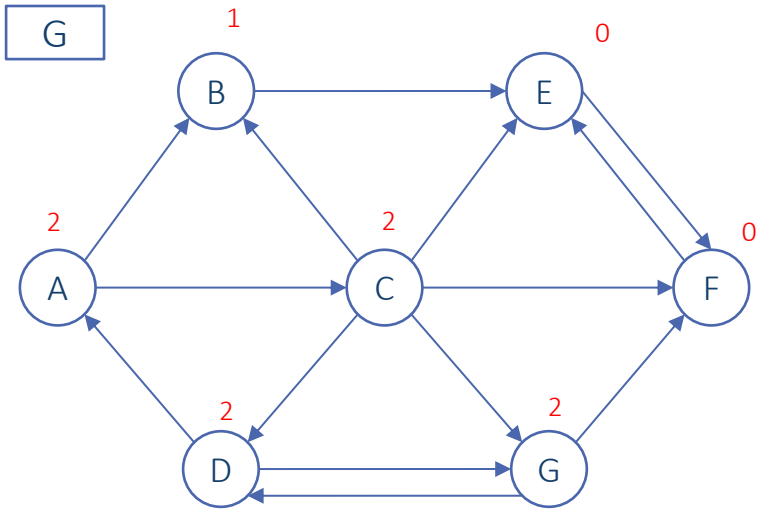


Call stack

search()
visit(E) 0
visit(F)
visit(F)
visit(E)
visit(B) 1
visit(B)

Pós-ordem invertida G^R
E F B A D G C

Exemplo



Call stack

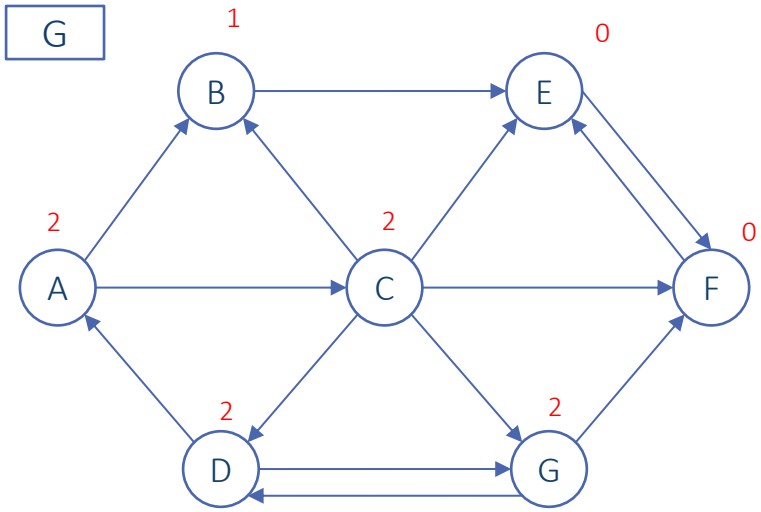
search()
  visit(E) 0
    visit(F)
    visit(F)
  visit(E)
  visit(B) 1
  visit(B)
  visit(A) 2
    visit(C)
      visit(D)
        visit(G)

Pós-ordem invertida G^R
E F B A D G C

Esta ordem só é relevante para o ciclo principal do método search.

Por isso quando visitamos A e C usamos a ordem convencional para visitar os seus vizinhos.

Exemplo



Call stack

search()
  visit(E) 0
    visit(F)
    visit(F)
  visit(E)
  visit(B) 1
  visit(B)
  visit(A) 2
    visit(C)
      visit(D)
        visit(G)
        visit(G)
      visit(D)
    visit(C)
  visit(A)
3

search()

Pós-ordem invertida G^R
E F B A D G C

Esta ordem só é relevante para o ciclo principal do método search.

Por isso quando visitamos A e C usamos a ordem convencional para visitar os seus vizinhos.

Complexidade Temporal

- Algoritmo de Kosaraju-Sharir pode ser dividido em 3 partes
  - Calcular G inverso G^R
  - Fazer procura em profundidade sobre G^R para determinar pós-ordem inversa
  - Procura em profundidade sobre G usando a ordem do passo anterior

- $T_{kosaraju\text{-}Sharir} = 3\ (E + V)$